



SAMSUNG

NOVA EXECUTIVE
EDUCATION
NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Samsung Innovation Campus

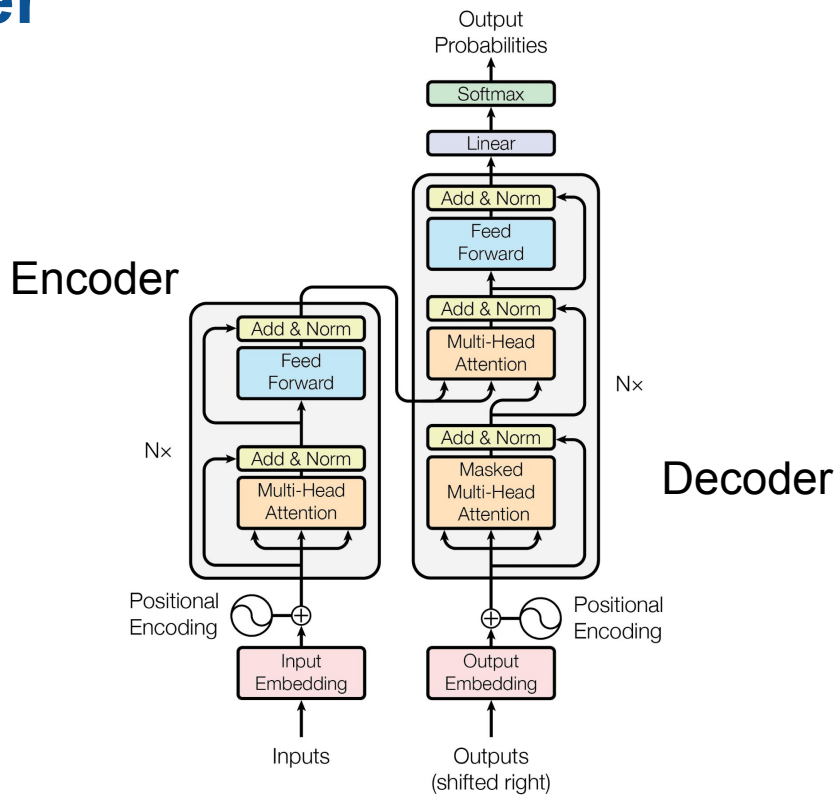
Chapter 9. Deep Learning Module

Chapter 9.

Transformer Part II: The Decoder



Transformer



Decoder: Here's your introduction

The decoder **generates** the output sequence one token at a time, using past generated tokens and (optionally) encoder outputs.

Auto-regressive

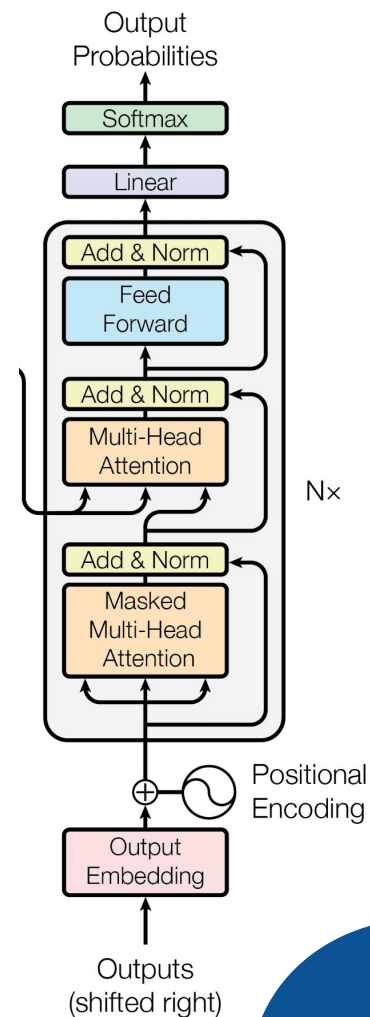
Predicts the next token

Causal

Cannot see into the future

Conditioned

Attend to encoder outputs



Decoder: The first steps

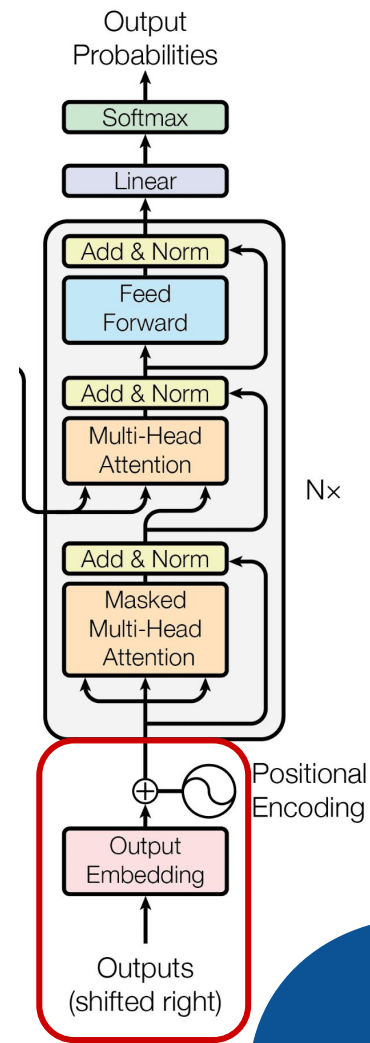
Input: the decoders previous output tokens.

- **Training:** Output tokens shifted right
- **Inference:** Previous predicted tokens

Given previous tokens, predict the next token

[BOS]: Beginning of Sequence Token

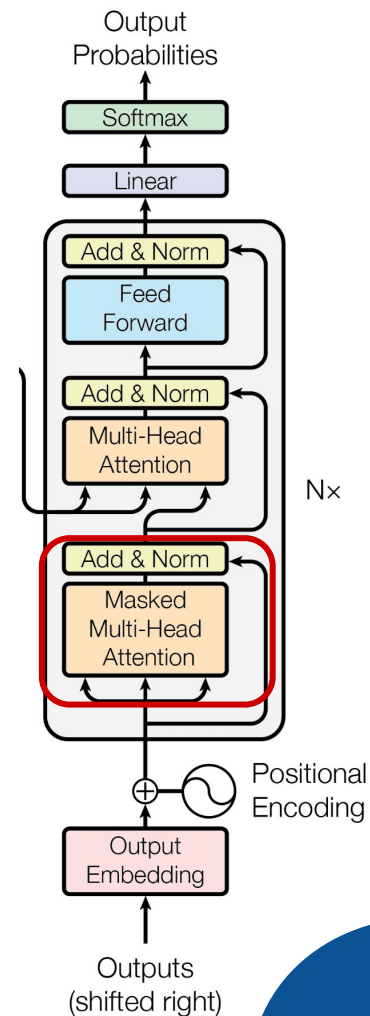
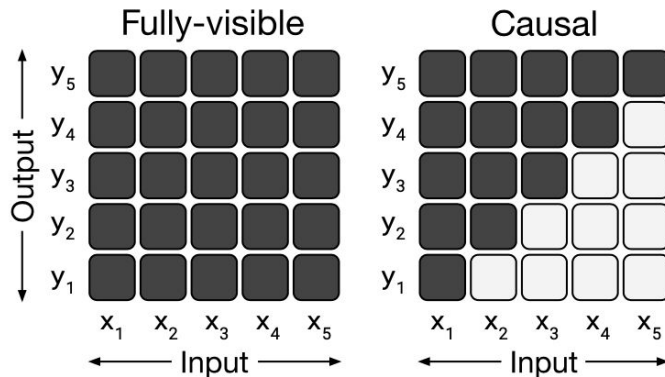
[EOS]: End of Sequence Token



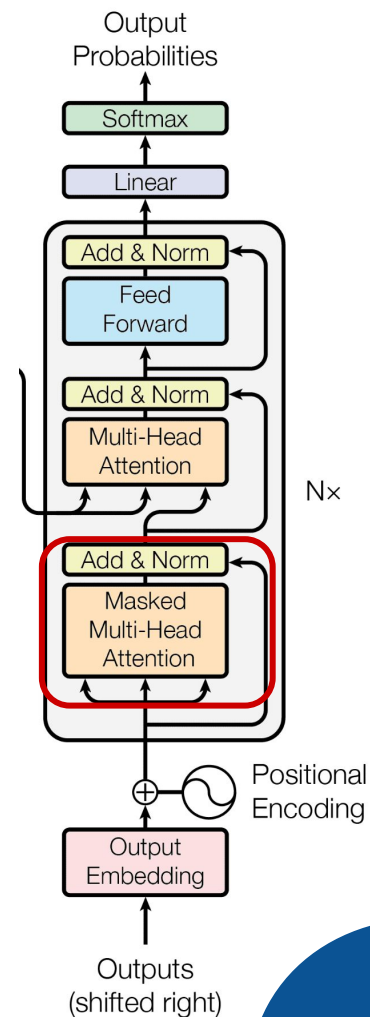
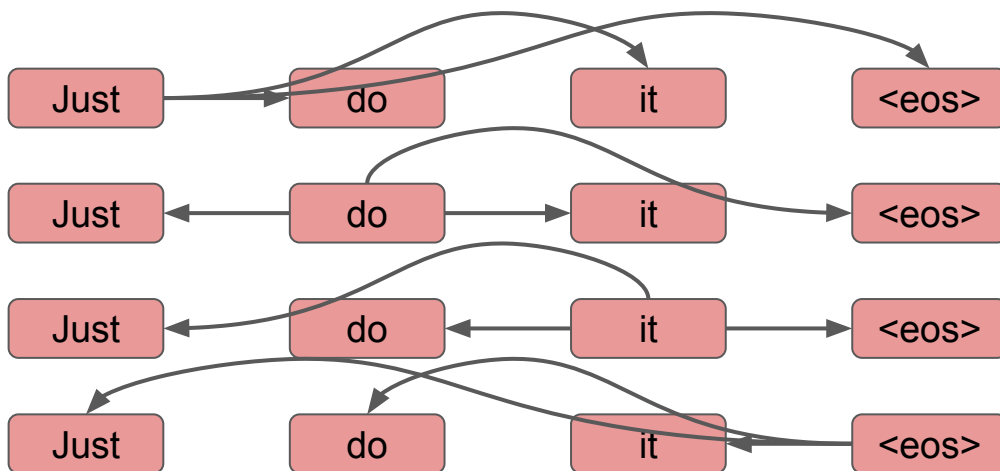
Decoder: Masked Self-Attention

Computes self-attention over the decoder's own tokens while applying a **causal (triangular) mask** so each token can only attend to itself and previous tokens.

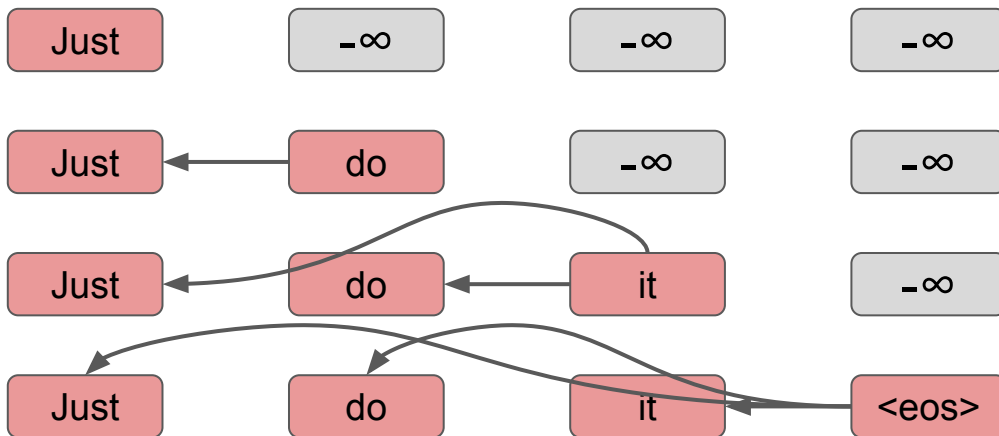
It doesn't allow the decoder to attend to future tokens!



Decoder: Masked Self-Attention

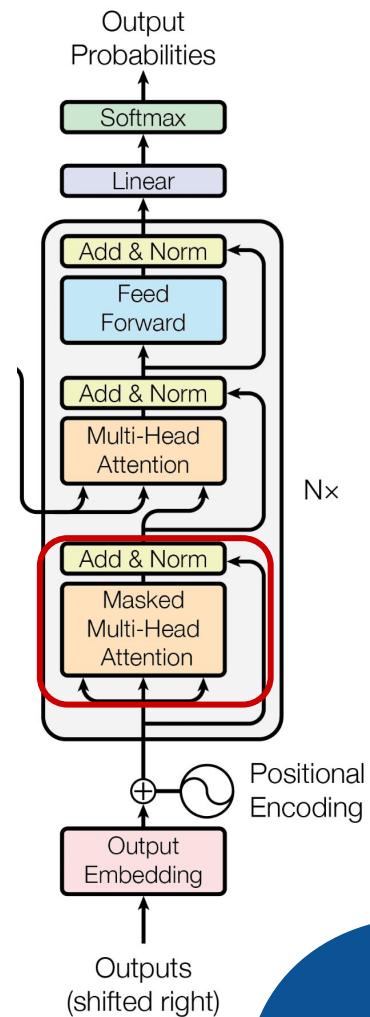


Decoder: Masked Self-Attention

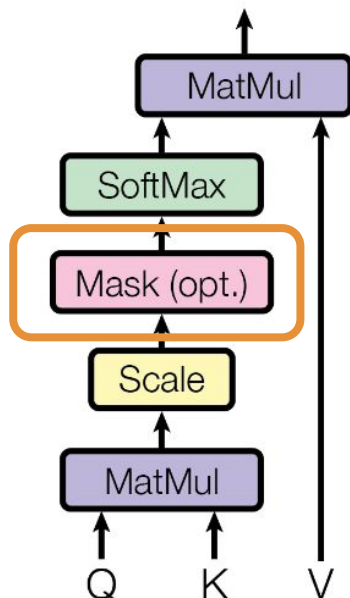


For token at position t attention to positions $> t$ is **not allowed!**
Disallows these into zero probabilities

Enforces left-to-right (CAUSAL) decoding!



Decoder: Masked Self-Attention



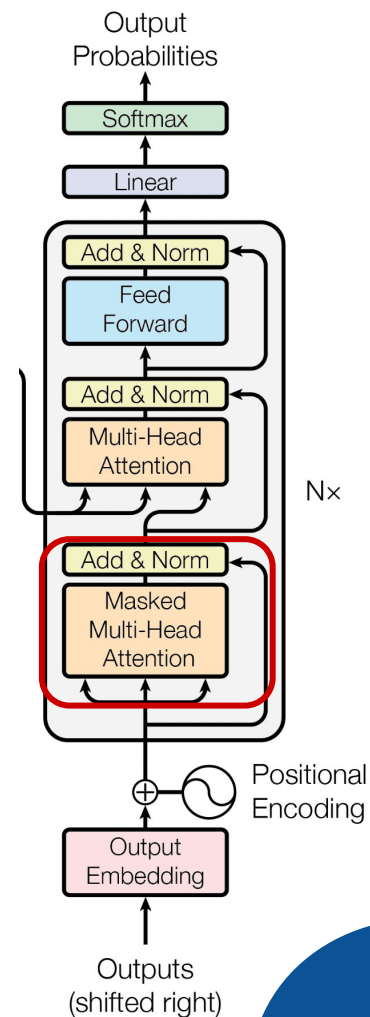
Attention Computation

In the **encoder** there's no masking!

But keep in mind:

Masking **controls which tokens can be attended to**, not what information the tokens contain!

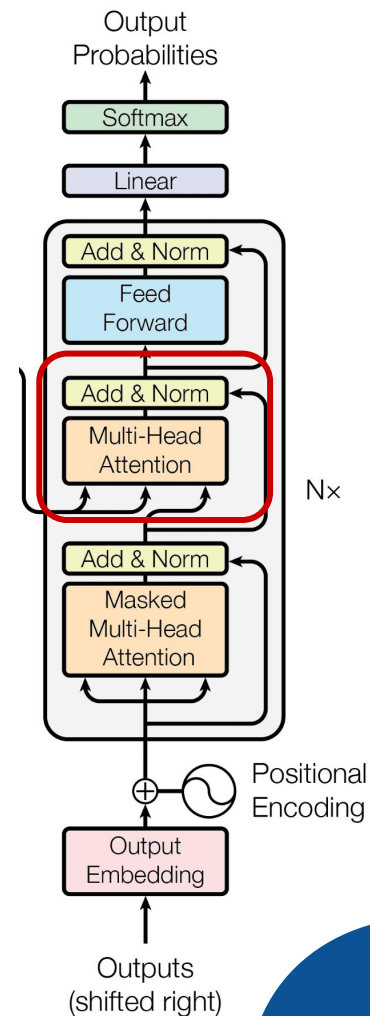
We are modifying attention weights, not token content!



Decoder: The Encoder has arrived!

So far we have seen two different types of attention that occur at different stages in the Transformer architecture:

- **Self-Attention**
 - Encoder
 - Q, K, V come from the **encoder inputs**
- **Masked Self-Attention**
 - Decoder
 - Q, K, V come from **decoder inputs**



Decoder: The Encoder has arrived!

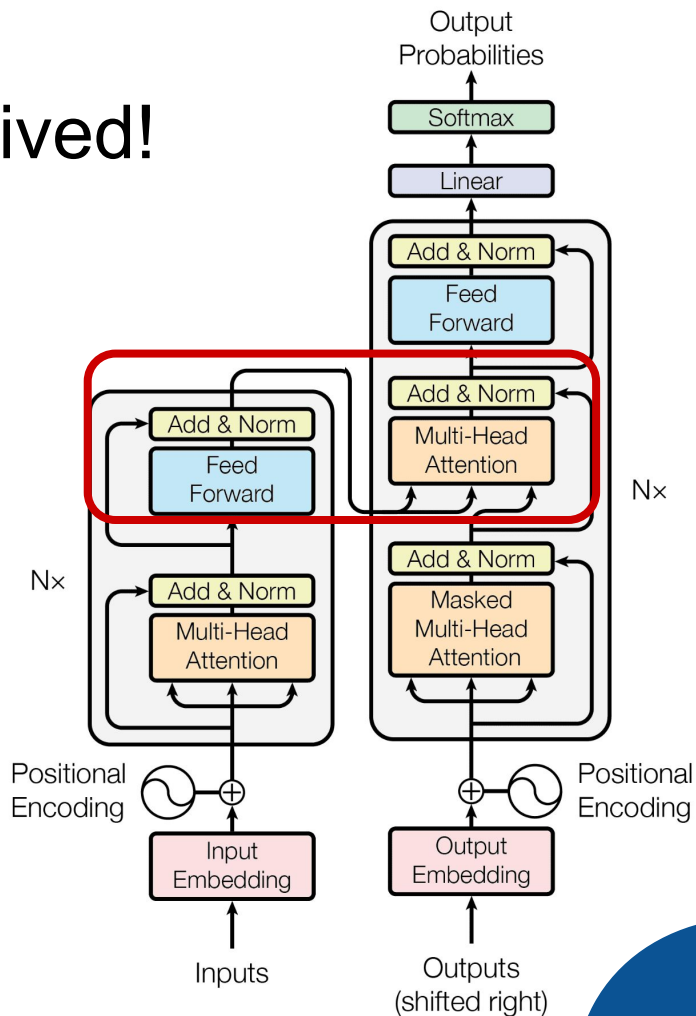
There's something missing...

Cross-Attention

Mechanism where one sequence (in the decoder) **attends to another** sequence (the encoder output).

Code Completion

Text Continuation



Decoder: Cross-Attention

It allows the decoder to **incorporate** relevant information from the encoder output when generating tokens.

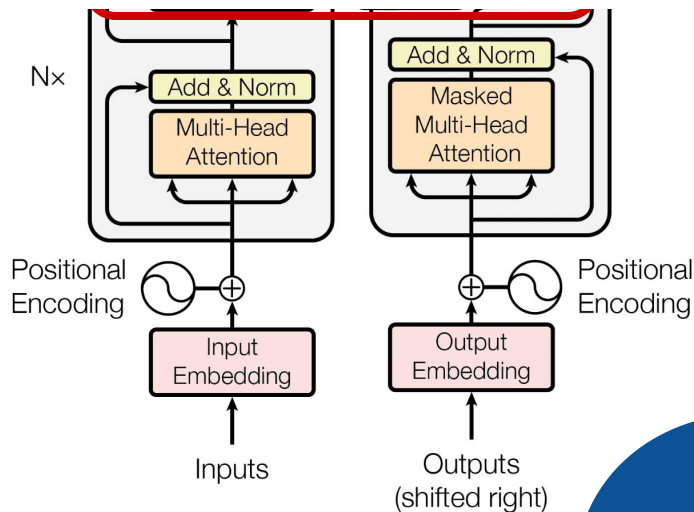
Q → Current Decoder layer

K → Final outputs of the Encoder

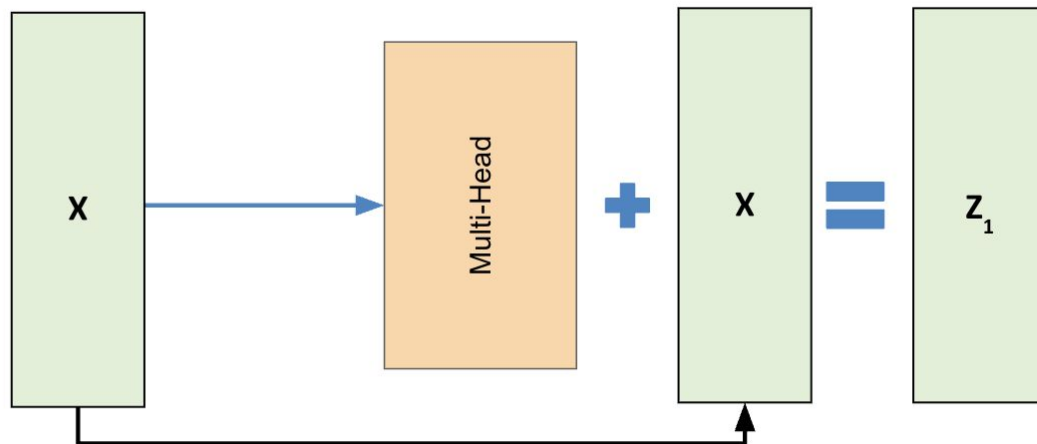
V → Final outputs of the Encoder

The decoder's queries attend to the encoder's keys and values to determine which parts of the encoded input should influence the current token prediction.

$$\text{CrossAttn}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}})$$



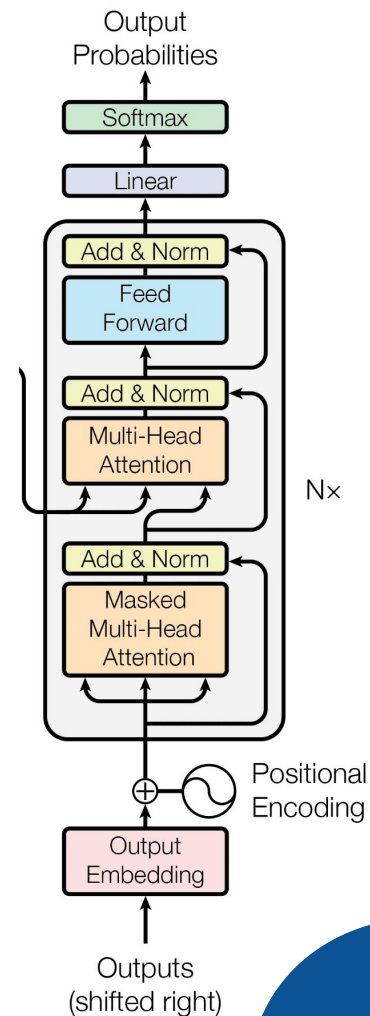
Decoder: The rest is history



Residuals: Helps retain the original input information (input and positional)

Normalization:

Stabilizes the learning process by ensuring the outputs have a consistent scale and distribution.

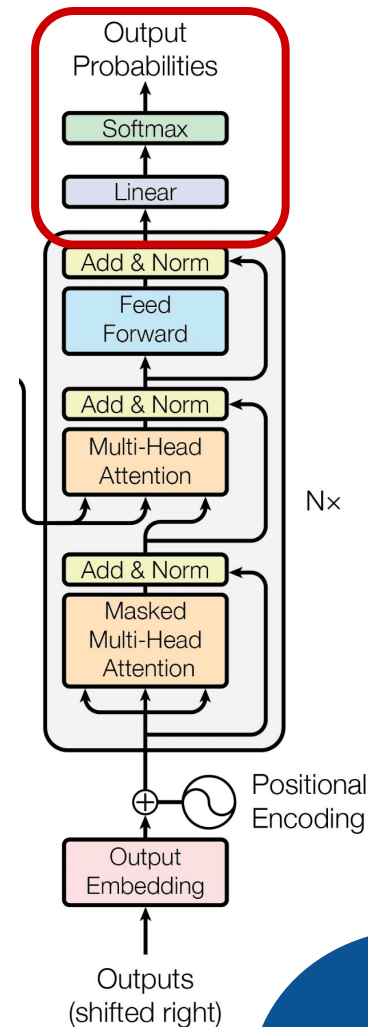


Decoder: Token prediction

We have the decoder outputs (a hidden representation of the sequence). These are fed into a **linear projection layer** which maps the **hidden state** into **vocabulary logits**.

The model is comparing every hidden state to every word in the vocabulary and scoring them on “how compatible” they are!

$$\text{logit}_{t,w} = h_t \cdot W_{\text{vocab}}[w]$$

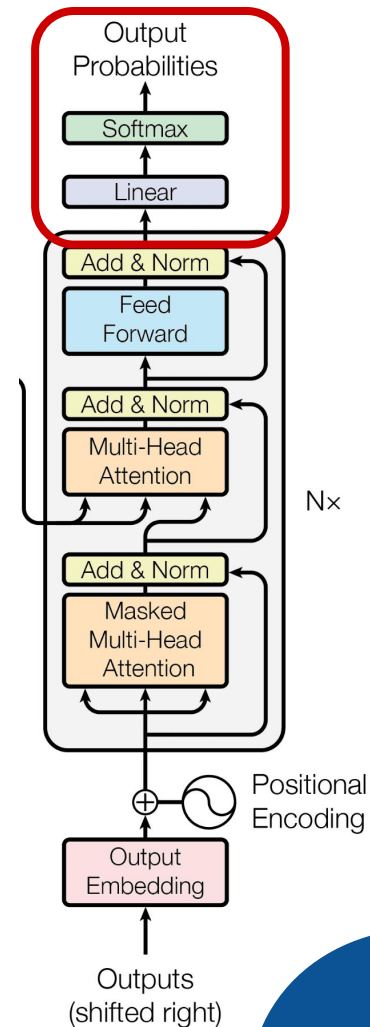


Decoder: Token prediction

The model then applies a softmax on top of the dot product in order to obtain the probabilities!

- The model **predicts a distribution** over the vocabulary
- Training minimizes negative log-likelihood of the true next token
- Implemented as **cross-entropy loss**

$$\mathcal{L}_t = -\log p(w_t^{\text{true}} \mid \text{context})$$

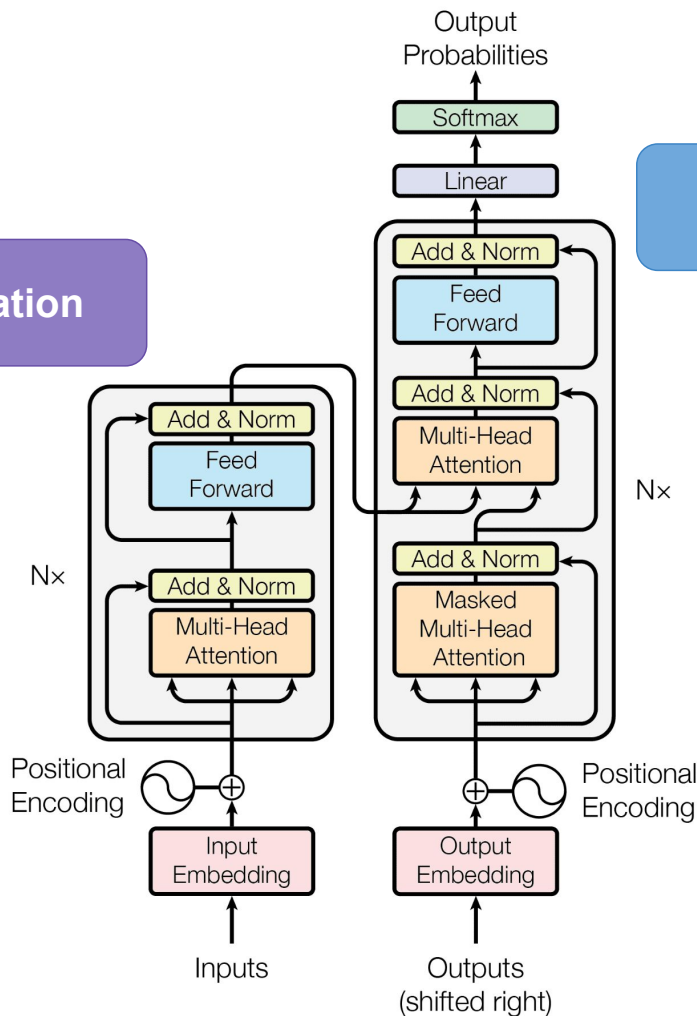


Representation

Input: Input Tokens

Output: Hidden States

Model can see all tokens!



Generation

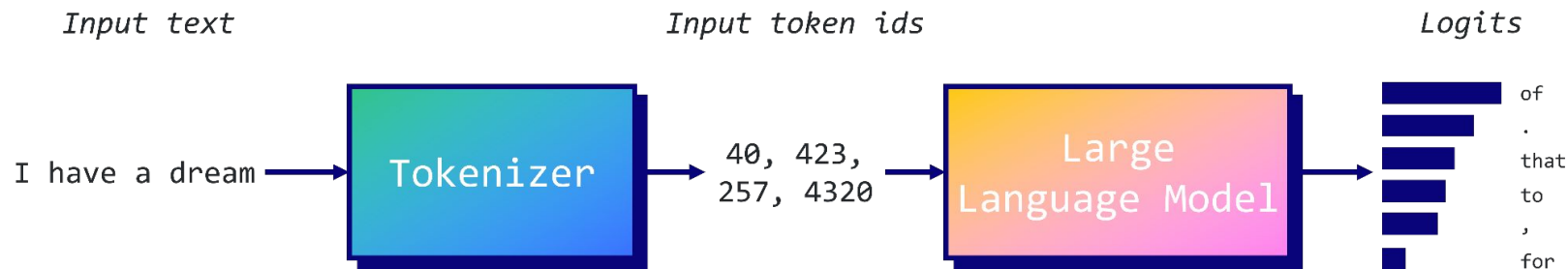
Input: Output tokens and hidden representations

Output: Output tokens

Model can only see previous tokens!

Decoder: Inference/Generation

This is what we've been (mostly) seeing so far!



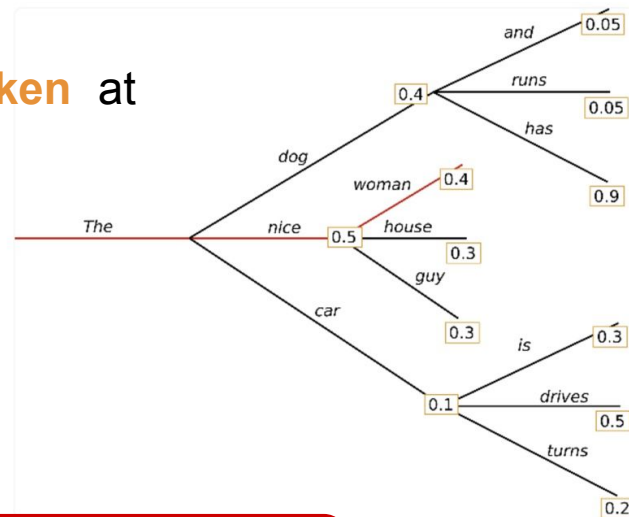
The token is generated, added to the end of the input and this is done until max length is reached or the [EOS] token is generated.

Decoder: Decoding Strategies

Greedy Search

Greedy search takes **the most probable token** at each step as the next token in the sequence.

It only retains the most likely token at each stage, discarding all other potential options!



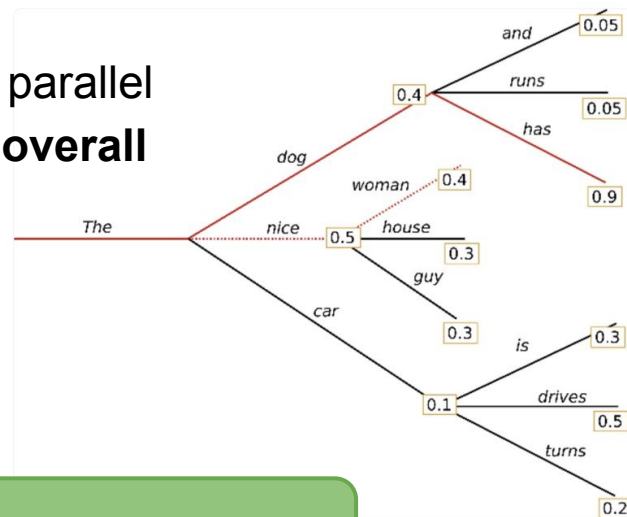
It ignores high probability words hidden behind low probability words

Decoder: Decoding Strategies

Beam Search

Beam search explores **multiple hypotheses** in parallel and selects the sequence with the **highest overall probability**.

It maintains a beam of size k and expands each candidate at every step

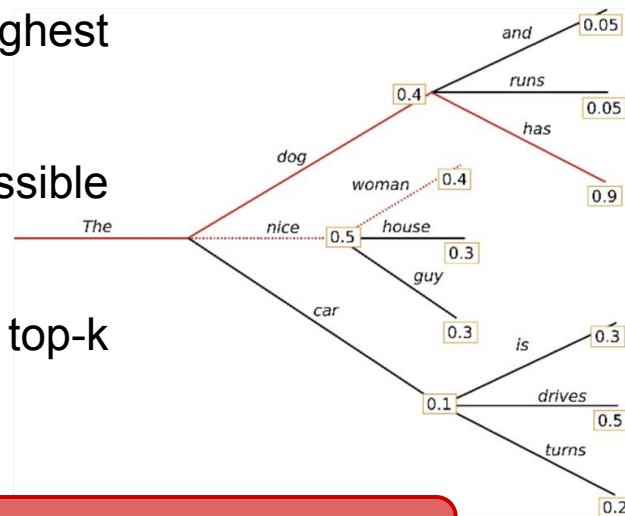


It often improves translation and caption quality

Decoder: Decoding Strategies

Beam Search

1. Select the top-k partial sequences with highest cumulative probability
2. Expand each sequence by predicting possible next tokens
3. Score all expanded sequences and keep the top-k overall
4. Repeat until <EOS> or max length



Computationally expensive

Decoder: Decoding Strategies

Top-k Sampling

Top-k sampling restricts the model to the **k most probable tokens**, then samples randomly from them. At each step, the model:

1. Selects the top-k highest-probability tokens
2. Renormalizes probabilities over this subset
3. Samples the next token from this restricted set

Increased diversity

Avoids nonsense tokens

Decoder: Decoding Strategies

Top-k Sampling

Temperature adjusts how confident or random the model's choices are.

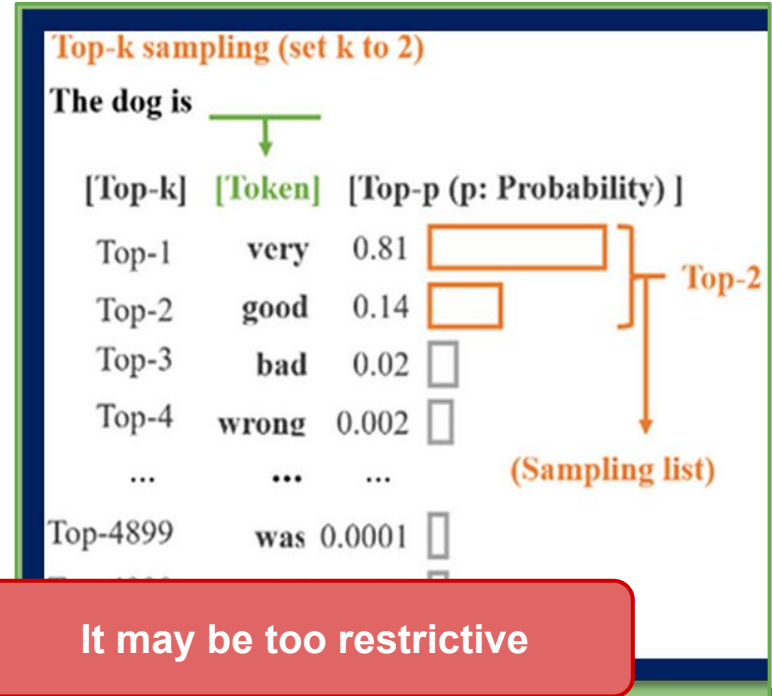
$T < 1.0 \rightarrow$ Sharper distribution

- More confident, conservative
- Closer to greedy behavior

$T = 1.0 \rightarrow$ No change

$T > 1.0 \rightarrow$ Flatter distribution

- More random, more diverse
- Increases creativity, but more errors



Decoder: Decoding Strategies

Top-p Sampling (Nucleus Sampling)

Top-p sampling selects the smallest set of tokens whose cumulative probability is at least p , then samples from that set. At each step, the model:

1. Sorts tokens by probability (highest to lowest)
2. Selects the minimal subset whose total probability $\geq p$
3. Samples from this dynamic set

Adapts to model confidence

Decoder: Decoding Strategies

Top-k sampling (set k to 2)

The dog is



[Top-k]	[Token]	[Top-p (p: Probability)]	
Top-1	very	0.81	Top-2
Top-2	good	0.14	
Top-3	bad	0.02	
Top-4	wrong	0.002	
...	...	...	
Top-4899	was	0.0001	
Top-4900	will	0.0001	
...	...	...	

(Sampling list)

Top-p sampling (set p to 0.96)

The dog is



[Top-k]	[Token]	[Top-p (p: Probability)]	
Top-1	very	0.81	Cumulative probability exceeds 0.96 (0.81+0.14+0.02 > 0.96)
Top-2	good	0.14	
Top-3	bad	0.02	
Top-4	wrong	0.002	
...	...	...	
Top-4899	was	0.0001	
Top-4900	will	0.0001	
...	...	...	

(Sampling list)

Decoder: Decoding Strategies

Deterministic

- Greedy Search
- Beam Search

Optimizes likelihood

Less diverse

Stochastic

- Top-k Sampling
- Top-p Sampling

More diverse

Less deterministic

Exercise Time



Chapter 9.

So what now?
The Future is here!

Large Language Models (LLMs)

What is a LLM?

A category of **foundation** models trained on extremely vast datasets in order to make them capable of understanding and generating natural language content.

**LLMs can generate human-like responses
based on context**

Question-Answering



Translation

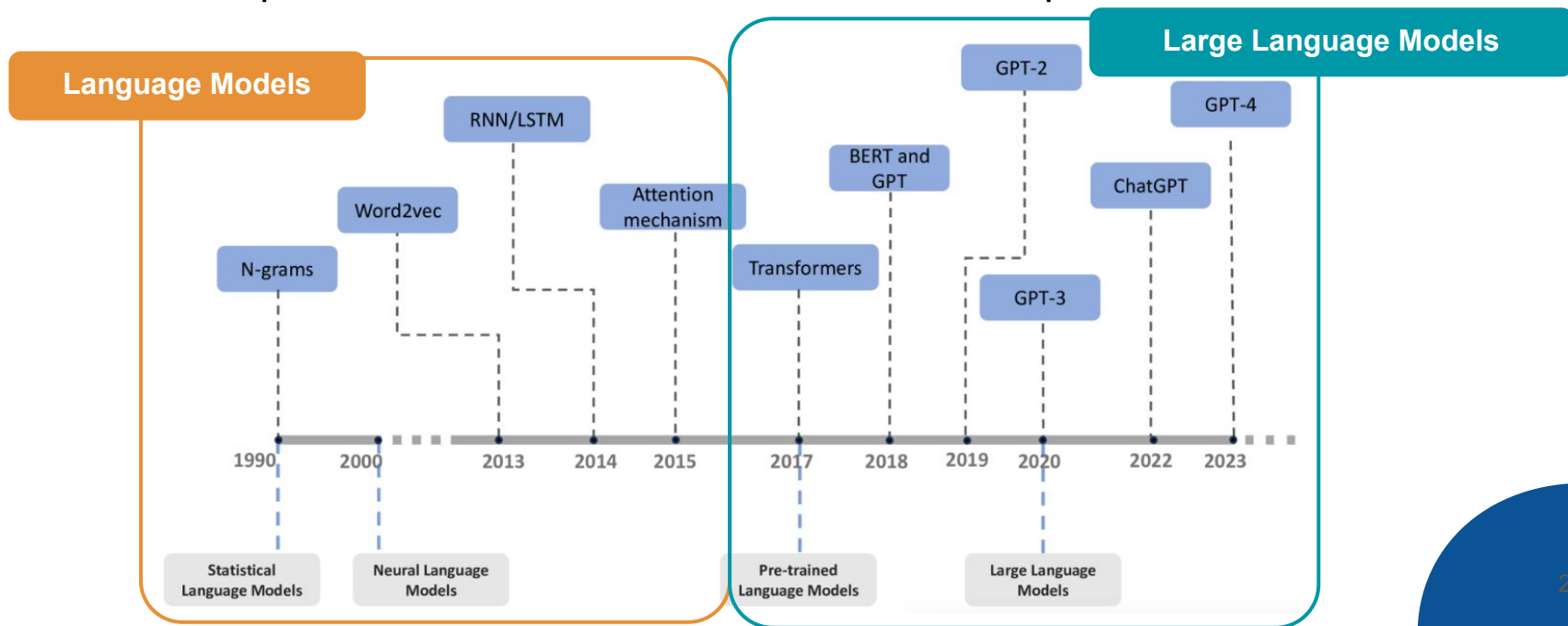


Text Summarization



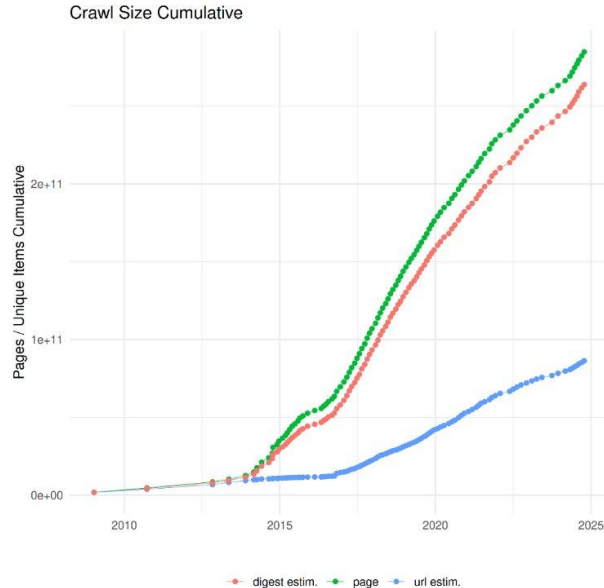
Language Modeling: The Concept

Definition: A probabilistic approach that involves predicting the next word in a sentence or sequence of words based on the context and previous words.



What made these models Large Language Models?

More Data!



Computer Power



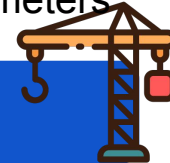
Requires massive GPUs/TPUs and distributed computing

More Parameters



From millions (BERT) to trillions (GPT-4) of parameters

Architecture



Transformers & Attention Mechanisms

What makes these models **Large Language Models**?

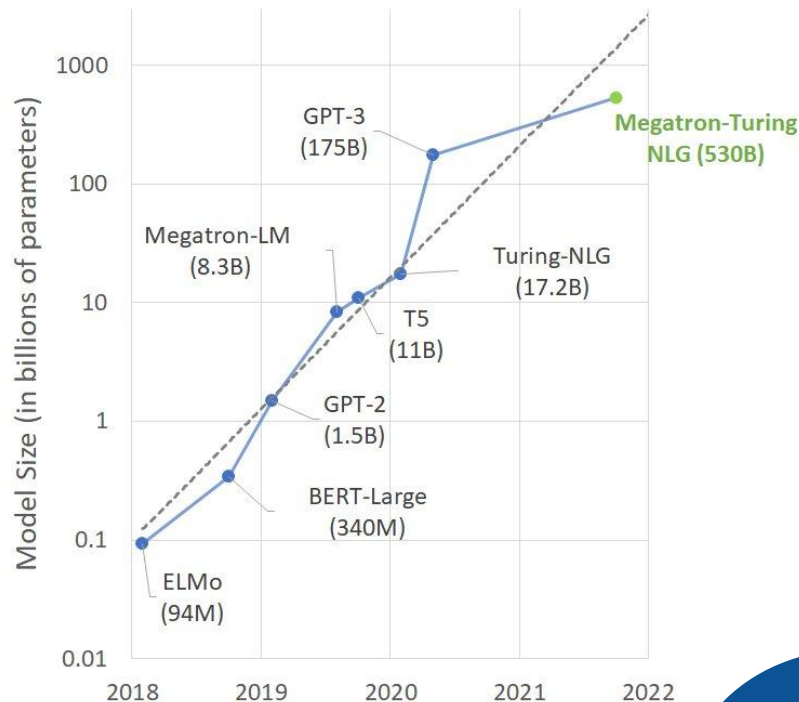
We are witnessing an **exponential growth** in model size!

More parameters

↑ Fluency, reasoning, and generalization

↑ Computational cost

Beyond Text: The next frontier is Multimodal LLMs



The Rise of Multimodal LLMs

MLLMs are models that process and integrate multiple types of **data modalities**.

Text

Images

Audio

Sensory Data



MLLMs: Why should we care?

These models enable “more natural” interactions and allow for advanced problem-solving capabilities that combine **different types of information!**

Document Analysis

Extracting both textual and visual information from documents (e.g., CV analysis)

Healthcare

Combining medical imaging with patient records to improve diagnostics

Autonomous Systems

Assists in navigation by integrating visual, auditory and textual cues (e.g., autonomous driving systems)

Content Creation & Digital Art

Generating or editing images based on textual inputs